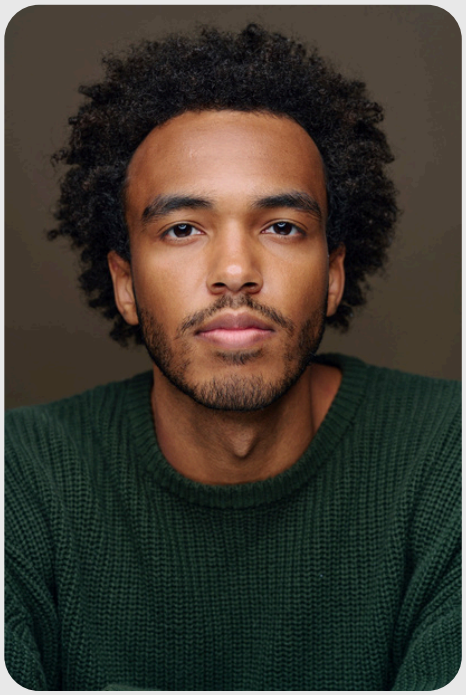
Step-by-Step: Learn How to Code Online & Land a $100,000+ Job ASAP!!

BASHIRI SMITH

Hey!



A LITTLE ABOUT ME

My name is Bashiri Smith.

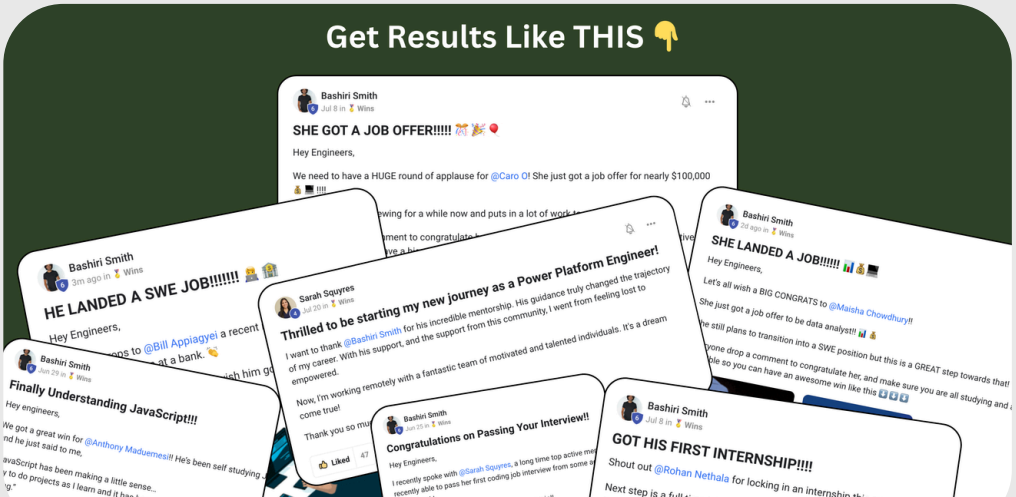
In just under 12 months & without a college degree or any work experience, I received 2 offers for more than $115,000 for my first job as a software engineer.

Since then, I have gone on to earn over $265,000p/y as a software engineer and I have also founded my own tech startup. (Interlade)

Throughout my journey to learn how to code, I've found that the unique information to ***ACTUALLY*** land a job is hard to come by regardless of if you go to college or a coding bootcamp.

My mission is to make it easy for anyone to change their life with coding!

This guide is for aspiring coders who want to earn $100k+ as a software engineer ASAP.



CONTENT

CHAPTER
01
Foundational Programming Concepts

CHAPTER
02
Frontend Development

CHAPTER
03
Backend Development

CHAPTER
04
Version Control

CHAPTER
05
Testing and Debugging

CHAPTER
06
Deployment and CI/CD

CHAPTER
07
Web Security

CONTENT

CHAPTER
08
Containerization

CHAPTER
09
Interviews & Soft Skills

CHAPTER
10
Resume Projects

CHAPTER
11
Need 1-on-1 Help?

01

Overview

To get the most out of this guide:

- Complete each step before moving to the next one
- Do not learn multiple things at once, STICK WITH ONE AT A TIME
- Don't cheat yourself and google/chat gpt your answers
- Code EVERYDAY for AT LEAST 2 hours a day. (4-6 if possible)
- Don't move on if you don't understand something. Figure it out!
- Watch this:

https://www.youtube.com/watch?v=NtfbWkxJTHw

01

Foundational Programming Concepts

What is Coding/Programming?

Coding, or programming, is the process of creating instructions for computers to follow. It involves writing code in a programming language, which is a special language understood by computers. This code tells the computer what actions to perform, enabling it to execute various tasks, from simple calculations to controlling complex systems. Think of it as writing a recipe that the computer follows to complete a specific task or solve a problem.

What are Programming Languages?

Programming languages are like special codes that tell computers what to do. Think of them as the secret languages that computer programmers use to make games, apps, and websites. Just like we use English or Spanish to talk to each other, programmers use these languages to give instructions to computers.

There are lots of different programming languages, and they can be grouped into a few types:

1. High-level languages are the easiest for people to understand. They are a bit like talking in full sentences. Examples include Python, Java, C++, and JavaScript. Programmers use these for making all sorts of things, like video games, apps, and websites.
2. Low-level languages are more like the computer's own language. They're harder for people to understand but let the programmer control the computer very closely. Assembly language is an example. It's used for special jobs that need the programmer to manage exactly how the computer works.
3. Markup languages aren't exactly programming languages, but they help set up how things look on websites or organize information. HTML, which helps create web pages, is an example.

Each programming language has its own set of rules on how to write and organize code, kind of like how English has grammar and spelling rules. When programmers write code, they have to follow these rules so the computer can understand them.

Choosing which programming language to use depends on what the programmer wants to make. Some languages are better for certain tasks than others, like how some tools are better for certain jobs.

01

Foundational Programming Concepts

What is an IDE?

An IDE, or Integrated Development Environment, is a software application that provides comprehensive facilities to computer programmers for software development. Think of it as a toolbox that contains all the tools a programmer needs to write, test, and debug their code all in one place. An IDE typically includes:

- A code editor: This is where you write your code. The editor often highlights syntax and can auto-complete some of your code, making it easier to read and write.
- A compiler or interpreter: This tool translates your written code into a language that computers can understand, either by compiling the code into executable files or by interpreting the code directly.
- A debugger: This tool helps you find and fix errors in your code. It allows you to run your program step by step, inspect variables, and understand what your code is doing at each step.
- Build automation tools: These tools automate common development tasks, like compiling your code into an executable application or deploying your web application to a server.

Some IDEs are designed for a specific programming language, such as Eclipse or IntelliJ IDEA for Java, while others, like Visual Studio Code or Atom, can be used with a wide variety of languages through the use of plugins. IDEs can greatly improve a programmer's productivity by providing all the necessary tools in a single application, making it easier to write, test, and debug software.

VSCode industry standard among most modern Frontend/Backend/FullStack Engineers (Free Download Below):

https://code.visualstudio.com/download

01

Foundational Programming Concepts

Terminal

As a software engineer, using the terminal or command line is essential because it provides a powerful and direct way to interact with your computer's operating system and software. It allows you to perform tasks more efficiently and quickly than through a graphical user interface (GUI). With the command line, you can execute commands for compiling code, managing files, accessing databases, and controlling version systems like Git. It's also indispensable for automation, scripting, and working with servers or development environments that don't have a GUI. Learning to use the terminal can significantly enhance your productivity, troubleshooting skills, and understanding of how systems work under the hood.

Command Line Crash Course:
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Understanding_client-side_tools/Command_line

01

Foundational Programming Concepts

- **Basic Syntax and Logic:** Familiarity with control structures, loops, functions, and data structures.
  - https://csx.codesmith.io/home

- Algorithms and Data Structures: Sorting, searching, stacks, queues, linked lists, trees, graphs, hash tables.
  - https://neetcode.io/courses/dsa-for-beginners

- Coding Challenges: Solve basic to intermediate-level coding problems like FizzBuzz, palindrome checks, and two-sum problems.
  - https://neetcode.io/practice

02 Frontend Development

What is HTML & CSS?
HTML (HyperText Markup Language) and CSS (Cascading Style Sheets) are foundational technologies used to create and design websites.

- HTML is the standard markup language used to create the structure of web pages. It allows you to define elements such as headings, paragraphs, links, images, and more. Essentially, HTML tells the web browser what each part of a website is (for example, a piece of text, an image, or a video), so the browser can display that content to users.
- CSS is a stylesheet language used to describe the presentation of a document written in HTML or XML (including XML dialects such as SVG). CSS defines how elements should be displayed on the screen, in terms of layout, colors, fonts, and more.

HTML is like the skeleton of a website—it helps make the basic structure and content, like text and pictures. CSS is like the website's clothing—it makes everything look nice by adding colors, styles, and layouts. Together, they help create websites that are both functional and attractive.

** These are listed in the order that I recommend you take them.
** COMPLETE THE ENTIRE COURSE, skipping steps will lead to a gap in knowledge down the road

Courses:
Free HTML & CSS course (MUST COMPLETE):
https://v2.scrimba.com/html-css-crash-course-c02l?via=bashifuirkashi

All Free Codecademy HTML & CSS Courses:
https://www.codecademy.com/catalog/language/html-css

02 Frontend Development

Why Javascript?

Learning JavaScript is a smart choice for several compelling reasons, especially if you're interested in web development, building interactive websites, or stepping into the vast field of software development. Here are key reasons why someone should learn JavaScript:

1. **Ubiquity in Web Development:** JavaScript is essential for web development. It's the only programming language that runs natively in web browsers, making it indispensable for creating interactive and dynamic web pages. Learning JavaScript means you can add interactive elements to websites, such as games, scrolling abilities, and dynamically updating content.
2. **Versatility:** JavaScript has grown beyond the browser. With environments like Node.js, you can write server-side code, develop desktop and mobile applications, and even get involved in machine learning and IoT (Internet of Things) projects. This versatility opens up a wide range of opportunities for JavaScript developers.
3. **High Demand for JavaScript Developers:** JavaScript consistently ranks as one of the most popular and in-demand programming languages in the job market. Companies, from startups to large enterprises, seek developers with JavaScript skills for various roles, ensuring a strong job outlook.
4. **Rich Ecosystem and Frameworks:** JavaScript's ecosystem is vast, with numerous libraries and frameworks (such as React, Angular, Vue.js) that simplify the development process, enhance productivity, and allow for the rapid creation of applications. These tools make JavaScript even more powerful and versatile for building complex applications.
5. **Easy to Learn for Beginners:** Compared to many other programming languages, JavaScript is relatively easy to start with. It has a straightforward syntax and can be run directly in web browsers, allowing beginners to quickly see the results of their code and experiment without needing complex development environments.
6. **Community and Resources:** The JavaScript community is one of the largest and most active in tech, offering extensive resources for learning and troubleshooting. From forums and blogs to online courses and meetups, there's an abundance of support available for those learning JavaScript.

Frontend Development

7. **Innovation and Future Growth:** JavaScript is at the forefront of modern web technologies and innovations. Learning JavaScript opens the door to working with cutting-edge technologies like augmented reality, virtual reality, and blockchain applications.
8. **Cross-Platform Development:** JavaScript enables developers to write code that runs across different platforms, including web, mobile (through frameworks like React Native), and desktop (with frameworks like Electron). This cross-platform capability means you can reach a wider audience without learning a new language for each platform.

Learning JavaScript not only boosts your skills in web development but also makes you a versatile developer capable of working on a wide range of projects across different platforms. Its importance in current and future technologies, along with the vast opportunities it presents, makes JavaScript a valuable language to learn and master.

Courses:

Free Intro to JS Course (MUST COMPLETE):
https://v2.scrimba.com/introduction-to-es6-cot?via=bashifuirkashi

Free Intermediate JS Course (MUST COMPLETE):
https://v2.scrimba.com/the-tricky-parts-of-javascript-coi?via=bashifuirkashi

Frontend Development

What is React?

React is a popular JavaScript library(see definition below) for building user interfaces, especially for web applications. Developed by Facebook, it enables developers to create reusable UI components, which makes developing complex, interactive, and dynamic web applications more efficient and easier to manage. Here are some reasons why learning React can be beneficial:

1. High Demand: React is one of the most popular front-end technologies in the industry. There's a strong demand for developers skilled in React, offering good job prospects and opportunities.

2. Reusable Components: React's component-based architecture lets you create reusable components. This means you can develop new applications faster and with less code, improving productivity and maintainability. Simplified: Reuse Pieces: With React, you can create small parts (like buttons or menus) and use them over and over again in different projects. This saves time and effort.

3. Strong Community Support: With a large and active community, React has a vast ecosystem of tools, libraries, and resources. This community support makes learning and problem-solving much easier.

4. Efficient Updates with Virtual DOM: React uses a Virtual DOM, which optimizes updates to the actual DOM in the browser, making it very efficient. This results in faster and smoother performance for user interfaces.

5. Versatile Use Cases: React can be used for a wide range of projects, from small personal websites to large-scale enterprise applications. It's also suitable for developing mobile applications with React Native.

6. Improves Job Skills: Learning React not only opens up job opportunities in web development but also helps you understand modern front-end development practices, making you a more versatile developer.

7. Easy to Learn for JavaScript Developers: If you already know JavaScript, picking up React can be relatively straightforward. Its simplicity and the fact that it's JavaScript-based make it accessible for those with a basic understanding of web development.

8. Integration with Modern Development Tools: React works well with modern development tools and methodologies, including module bundlers like Webpack, development environments like Node.js, and testing frameworks.

9. Foundation for Related Technologies: Mastering React provides a good foundation for learning related technologies, such as React Native for mobile app development, making it a valuable skill in your toolkit.

In summary, learning React is a wise investment for anyone looking to advance in the field of web development. Its popularity, efficiency, and the rich ecosystem make it a key technology for developing modern web applications.

Courses

** Please Complete In Order!
Free React Courses (MUST COMPLETE):
https://v2.scrimba.com/learn-react-coe?via=bashifuirkashi

Free Codecademy React Hooks (MUST COMPLETE):
**React hooks will be used on the job soooo much. Learn This!
https://www.codecademy.com/learn/learn-react-hooks

Free React Course (MUST COMPLETE):
https://react.dev/learn/tutorial-tic-tac-toe

03 Backend Development

What is Node?

In the context of backend development, Node.js is a powerful and popular environment that allows developers to use JavaScript to write server-side code. This means with Node.js, you can build the part of websites and applications that runs on the server, managing tasks like connecting to databases, handling user requests, and serving responses to the client (such as web browsers).

Before Node.js, JavaScript was primarily used for creating interactive features on web pages (frontend development). Node.js extended JavaScript's capabilities to the server side, enabling developers to use a single programming language across the entire development stack. This uniformity can simplify the development process, making it easier for developers to switch between frontend and backend work.

Node.js is designed to be efficient and scalable, capable of handling many connections simultaneously. It achieves this through an event-driven, non-blocking (asynchronous) I/O model, which makes it particularly well-suited for building applications that require real-time data processing, like live chat applications, online gaming, and collaboration tools.

In summary, in backend development, Node.js is used to write the code that runs on the server, handling the logic and data processing that supports the user-facing side of web applications and websites. Its use of JavaScript for server-side programming, efficiency, and scalability make it a popular choice among developers for building a wide range of web applications.

Courses

Free Codecademy Node.js Course:
https://www.codecademy.com/learn/learn-node-js

03 Backend Development

Database Coursework
Understanding databases is crucial since they're where all the data your app creates and uses is stored. There are two main types of databases: relational and non-relational. Let's break them down:

Relational Databases
Relational databases are like sophisticated spreadsheets. They store data in tables, which are a bit like sheets in a spreadsheet, with rows and columns. Each row is a record (like an entry in your contact list), and each column is an attribute of that record (like a person's name, phone number, etc.). These tables can be linked to each other; that's why they're called "relational" — because you can relate data from one table to data in another. This is really useful for organizing complex data structures.
Most companies use relational databases because they're reliable, have been around for a long time, and support complex queries and transactions. They're great for applications where data integrity and relationships between different sets of data are important, like banking systems or customer relationship management (CRM) systems.
Popular Technologies for Relational Databases:
- PostgreSQL: Known for its reliability, flexibility, and support for advanced data types. It's open-source and can handle complex queries, making it a favorite for many developers and companies.
- MySQL: Another popular open-source option, widely used for web applications. It's known for its ease of use and speed.
- Oracle Database: A powerful commercial database system that offers advanced features for large-scale applications, often used by large corporations.

Non-Relational Databases

Non-relational databases, or NoSQL databases, are more like a flexible storage locker. Instead of storing data in tables, they can store it in various ways, such as key-value pairs (think of a coat check ticket where your coat is the data and the ticket is the key), documents (which are like more complex and nested versions of key-value pairs, great for hierarchical data), or graphs (which are superb for mapping relationships, like social networks).

These databases are designed to be more flexible and scalable than relational databases, making them a good choice for applications with large amounts of data or for data that doesn't fit neatly into tables, such as JSON documents or real-time analytics.

Popular Technologies for Non-Relational Databases:

- MongoDB: A document database that stores data in JSON-like formats, making it a great choice for applications that require rapid development and scaling.
- Redis: A key-value store known for its speed and often used for caching to improve application performance.
- Cassandra: Designed for handling large amounts of data across many commodity servers, providing high availability with no single point of failure.

Highlight on PostgreSQL

PostgreSQL stands out particularly for full-stack developers because it combines the robustness and reliability of traditional relational databases with some of the flexibility of non-relational databases. It supports advanced data types like JSON, allowing for efficient storage and querying of more complex, unstructured data. This makes PostgreSQL an incredibly versatile database that can be used across a wide range of applications, from small startups to large enterprises. Its open-source nature means it has a strong community and lots of resources to help you learn and troubleshoot. For a full-stack web engineer, mastering PostgreSQL can provide a strong foundation for handling almost any data management requirement your applications may have.

Free Codecademy SQL Course (MUST COMPLETE):
https://www.codecademy.com/learn/intro-to-sql
Free Codecademy NoSQL- MongoDB Course (MUST COMPLETE):
https://www.codecademy.com/learn/learn-mongodb

04

Version Control

What is Git?

Git is a version control system that software engineers use to track changes in their code over time. Think of it as a history book of your code, where you can save different versions of your projects as you make changes. This way, if something goes wrong, you can easily go back to a previous version. Git also makes it easier for multiple people to work on the same project by allowing them to merge their changes together. Essentially, Git helps manage and collaborate on code projects, making it easier to fix mistakes and work together.

- Git: Basic commands (clone, commit, push, pull), branching, merging, and workflows.

Free Courses (MUST COMPLETE):
https://www.codecademy.com/learn/learn-git-introduction
https://gitimmersion.com/lab_48.html

05 Testing and Debugging

Frontend Testing:

- Jest and React Testing Library for unit and integration tests in React.
  - https://www.codecademy.com/learn/learn-react-testing
  - https://www.youtube.com/watch?v=zuKbR4Q4280
  - https://www.classcentral.com/course/tau-jest-testing-tutorial-43742

Backend Testing:

- Supertest for Node.js.
  - https://github.com/ladjs/supertest#readme
  - https://www.codecademy.com/learn/learn-testing-for-web-development-server-testing

06

Deployment and CI/CD

Deployment and CI/CD (Continuous Integration/Continuous Delivery) are essential practices for modern software development, especially in full-stack engineering.

Deployment is the process of taking your application or website from a development environment (like your computer) and making it accessible to users on the internet. This typically involves hosting your app on a server or cloud service, where it can be accessed 24/7. Tools like Vercel, Netlify, and Heroku make this process straightforward by allowing developers to push their code online with just a few steps. These platforms handle the complex setup, such as configuring servers and managing resources, so you can focus on building your app rather than setting up infrastructure.

CI/CD stands for Continuous Integration and Continuous Delivery, and it's a set of automated practices designed to streamline and improve the software development workflow.

Continuous Integration (CI) means regularly integrating code changes into a shared repository, where automated tests can quickly catch any bugs or conflicts.

Continuous Delivery (CD) then ensures that these code changes are ready to be deployed to production with minimal effort. Platforms like CircleCI make CI/CD accessible by automatically building, testing, and preparing your app for deployment every time you make a change, so updates can go live faster and with more confidence.

Deploying Apps:

- Vercel
  - https://vercel.com/docs/getting-started-with-vercel
- Netlify
  - https://www.codecademy.com/learn/deploying-with-netlify
- Heroku
  - https://www.codecademy.com/learn/deploying-with-heroku

CI/CD:

- Circle CI
  - https://circleci.com/ci-cd/

07

Web Security

Web security is a crucial area for anyone building websites or applications, as it ensures that both the site and its users are protected from potential threats. Good web security practices prevent attackers from exploiting weaknesses in your application that could expose sensitive data, take down your site, or cause other serious issues.

Some common web security threats include Cross-Site Scripting (XSS), where attackers inject malicious code into a webpage viewed by other users; SQL Injection, where malicious code is sent into database queries to retrieve or modify data without permission; and Cross-Site Request Forgery (CSRF), where unauthorized commands are transmitted from a user that the web application trusts. Secure applications use practices like validating user input, employing HTTPS encryption, and managing user sessions securely to prevent these attacks.

With strong web security practices, developers protect not only their apps but also the users who rely on them. By learning about potential vulnerabilities and implementing defenses against them, you can build applications that are more resilient to attacks and better equipped to handle sensitive information.

Web Security Resources:

- Mozilla's Guide to Web Security
  - https://infosec.mozilla.org/guidelines/web_security

- Web Application Security for Software Engineers - Educative
  - https://www.educative.io/courses/web-application-security-everyday-software-engineer?eid=5082902844932096

08

Containerization

Containerization is like creating a compact, portable environment where your application can run independently from the specifics of the host system. Imagine your app as a product with specific parts, tools, and instructions required to work properly. Traditionally, when you move this product to another "workshop" (another computer or server), you might find the tools or parts there are slightly different, causing compatibility issues or unexpected behavior. Containerization solves this by bundling your application with everything it needs into a single unit – the container – that runs the same way no matter where you deploy it.

Docker is the leading tool in containerization and lets you define this environment in a configuration file called a Dockerfile. Inside the Dockerfile, you specify everything your application needs to run: the operating system, libraries, dependencies, and configurations. Once built, Docker packages these files and instructions into an image, which you can then use to create containers. Think of the image as a blueprint and the container as an actual instance of your application.

Key benefits of containerization with Docker include:

1. **Consistency Across Environments**: Since containers encapsulate everything your app needs, the same container can run on a developer's laptop, a testing server, or a production environment without requiring special configurations.
2. **Isolation**: Each container is isolated, meaning that it doesn't interfere with other containers or the host system, making it safer and easier to manage.
3. **Scalability**: Containers are lightweight, which means you can run many on a single machine or quickly scale up or down in response to demand.
4. **Efficiency**: Containers share the same OS kernel and resources more effectively than virtual machines, making them faster to start up and less resource-intensive.

By using Docker, you also unlock tools like **Kubernetes** for managing large numbers of containers across clusters of machines. Kubernetes automates the deployment, scaling, and management of containerized applications, making it ideal for production environments where applications need to be reliable and scalable.

Free Resources for Learning Containerization:

- Introduction to Containers and Docker (freeCodeCamp YouTube Tutorial)
  - https://www.youtube.com/watch?v=kTp5xUtcalw
- Docker's Getting Started Guide
  - https://docs.docker.com/get-started/

09

Interview & Soft Skills

Mastering Soft Skills and Interview Techniques for Software Engineering Roles

In software engineering, technical expertise is crucial, but soft skills are equally essential for long-term success. Hiring managers increasingly value candidates who demonstrate strong communication, problem-solving abilities, adaptability, and teamwork skills. Cultivating these skills can set you apart and make you a more effective team player and leader. Here's a guide on how to showcase these skills effectively during interviews and tips on refining them in practice.

1. **Preparing for Behavioral Interview Questions**

Behavioral questions help interviewers understand how you approach challenges, work within teams, and manage your work. Prepare using the STAR method: Situation, Task, Action, and Result. For example, if asked about a time you resolved a conflict, outline a specific situation, the challenge or task, the actions you took, and the results. Use concrete examples from past experiences—whether personal projects, internships, or previous jobs—that illustrate your ability to handle similar situations in the workplace.

Example Question: "Describe a time when you faced a challenging project deadline."
Answer (STAR): Briefly describe the project and its importance (Situation), your role and responsibilities (Task), the steps you took to prioritize tasks or collaborate (Action), and the final result, focusing on any positive outcomes.

2. **Demonstrating Communication Skills**

Effective communication in software engineering involves not only explaining your work clearly but also understanding others' perspectives. During interviews, practice summarizing complex technical details in simple terms that a non-technical person could understand. This will showcase your ability to convey information across different audiences, a skill that's vital when working with cross-functional teams, stakeholders, or clients.

Example Exercise: Practice explaining a technical concept like "RESTful APIs" or "object-oriented programming" in layman's terms. It's not just about simplifying terms but also ensuring the listener grasps why the concept matters.

In the interview, actively listen to the questions, ask for clarification when needed, and structure your answers logically. Showing that you listen and respond thoughtfully is part of good communication.

Interview & Soft Skills

3. Problem-Solving and Critical Thinking

Problem-solving is fundamental in software engineering. In interviews, technical questions often test this skill, but so do situational questions that involve hypothetical challenges. Here, outline your thought process and decision-making framework.

Example Approach:
- Define the Problem: Explain how you identify and understand the core of a problem.
- Break Down the Solution: Share your approach to breaking a problem into manageable parts.
- Evaluate and Adapt: Discuss how you choose tools or methods and adapt when encountering roadblocks.

For technical problem-solving, don't rush to answer. Take a moment to structure your response or even ask if you can write down key points before explaining them. This shows the interviewer that you approach problems systematically.

4. Adaptability and Openness to Feedback

Adaptability is about staying flexible in the face of change, which is especially valuable in fast-paced tech environments. Interviewers may ask about times when you had to learn a new skill quickly or work under new circumstances.

When answering questions about adaptability:
- Emphasize Continuous Learning: Talk about any recent technologies or frameworks you've learned.
- Show Willingness to Change: Share an example where you modified your approach based on new information or feedback.

Demonstrating a growth mindset also involves openness to feedback. If an interviewer provides a hint during a technical challenge, thank them, adapt your approach, and integrate their advice smoothly.

Interview & Soft Skills

5. Teamwork and Collaboration

Software engineers frequently work in teams, often with members from diverse backgrounds. Employers look for candidates who can contribute positively to team dynamics and work harmoniously in collaborative settings.

In interviews, emphasize:
- Empathy: How you value and respect others' ideas, even when they differ from your own.
- Contribution and Ownership: Share examples of taking initiative in group projects while also supporting others.
- Conflict Resolution: Describe a time when you resolved or helped diffuse a conflict, focusing on your collaborative approach to problem-solving.

Being a good team player means not only being a leader when necessary but also supporting and empowering others. Displaying humility and a genuine interest in collaborative success can go a long way.

6. Bringing It All Together

Conclude your interview by summarizing both your technical and soft skills. Reinforce how your communication, problem-solving abilities, adaptability, and teamwork make you a well-rounded candidate ready to tackle challenges and contribute effectively.

Finally, ask questions at the end of the interview to show interest and enthusiasm. Inquire about team dynamics, the role's collaborative aspects, or the company's approach to professional growth. Thoughtful questions underscore your interest in fitting in well with the team and contribute to a positive, memorable impression

Free Material:
- https://www.youtube.com/watch?v=DY3ZSnCV_R4

What is a "Resume Quality" Project

Qualities of a Resume-Level Project

- Complexity: Demonstrates a high level of technical skill and problem-solving ability.
- Scalability: Shows consideration for scalability and performance.
- Usability: Has a well-designed user interface and user experience.
- Documentation: Includes comprehensive documentation for both users and developers.
- Testing: Includes unit tests, integration tests, and possibly end-to-end tests.
- Version Control: Uses version control with a well-documented commit history.
- Deployment: Is deployed and accessible online, showcasing real-world usability.

Checklist to Pick Your Own Resume-Level Project

- Identify a Problem: Choose a real-world problem that interests you.
- Research Solutions: Investigate existing solutions and identify gaps or areas for improvement.
- Define Scope: Clearly define the scope of your project to ensure it is achievable within a reasonable timeframe.
- Plan Features: List the features your project will have and prioritize them.
- Design Architecture: Plan the architecture of your project, considering scalability and maintainability.
- Develop Incrementally: Build your project incrementally, testing each feature as you go.
- Document Thoroughly: Write comprehensive documentation for your project.
- Test Rigorously: Implement and run tests to ensure your project is robust and reliable.
- Deploy and Maintain: Deploy your project online and maintain it, fixing bugs and adding improvements as needed.

Find & Work with Teammates

The Importance of Teamwork

Working on a team is crucial for creating a resume-worthy project. Teamwork not only simulates real-world software development environments but also offers numerous benefits:

- Effective Communication: Learn to articulate your ideas and understand others'.
- Diverse Perspectives: Gain insights and solutions from different viewpoints.
- Task Division: Leverage individual strengths to tackle various project components efficiently.

Finding Teammates in a Community

To help you find and team up with other members of the community, follow these steps:

1. Join this FREE software engineering community: https://www.skool.com/become-a-software-engineer/about
2. Introduce Yourself
   - Post a brief introduction in the community (under the TeamUp category), including your skills, interests, and the type of project you want to work on. This helps others with similar goals and complementary skills find you.
3. Create a Project Board
   - We have set up a dedicated space where you can pitch your project ideas and look for teammates. Each post should include:
     - Project Description: What the project is about.
     - Required Skills: What skills are needed to complete the project.
     - Commitment Level: The expected time and effort.
4. Participate in Networking Events
   - Join our regular virtual meetups and networking events. These sessions provide opportunities to discuss your projects and form teams. Keep an eye out for announcements about these events.
   - Meetups to be scheduled!
5. Utilize Collaboration Tools
   - We recommend using tools that facilitate teamwork, such as:
     - GitHub: For version control.
     - Trello: For project management.
     - Slack: For communication.

By following these steps and engaging with your peers, you'll be well on your way to creating a project that stands out on your resume. Teamwork makes the dream work!

Get Your Project Idea Verified

Before you start working on your resume-level project, it's crucial to ensure that your idea meets the necessary criteria and will effectively showcase your skills to potential employers. To get your project idea verified, you can reach out to me directly.

How to Verify Your Project Idea

1. Outline Your Idea: Write a brief outline of your project idea, including the problem it addresses, the features you plan to implement, and the technologies you intend to use.
2. Contact Me: You can DM me or use the link on the last page to book a call with me.
3. Receive Feedback: I will review your project idea and provide feedback, suggestions, and verification to ensure it aligns with the qualities of a resume-level project.

Why Verification is Important

Verifying your project idea helps you:
- Ensure your project demonstrates a high level of technical skill.
- Align your project with industry standards and expectations.
- Receive guidance on best practices and potential improvements.
- Increase your chances of impressing potential employers with a well-crafted project.

Don't hesitate to reach out for verification; it's a critical step in your journey to becoming a successful software engineer.

Need 1-on-1 Help?

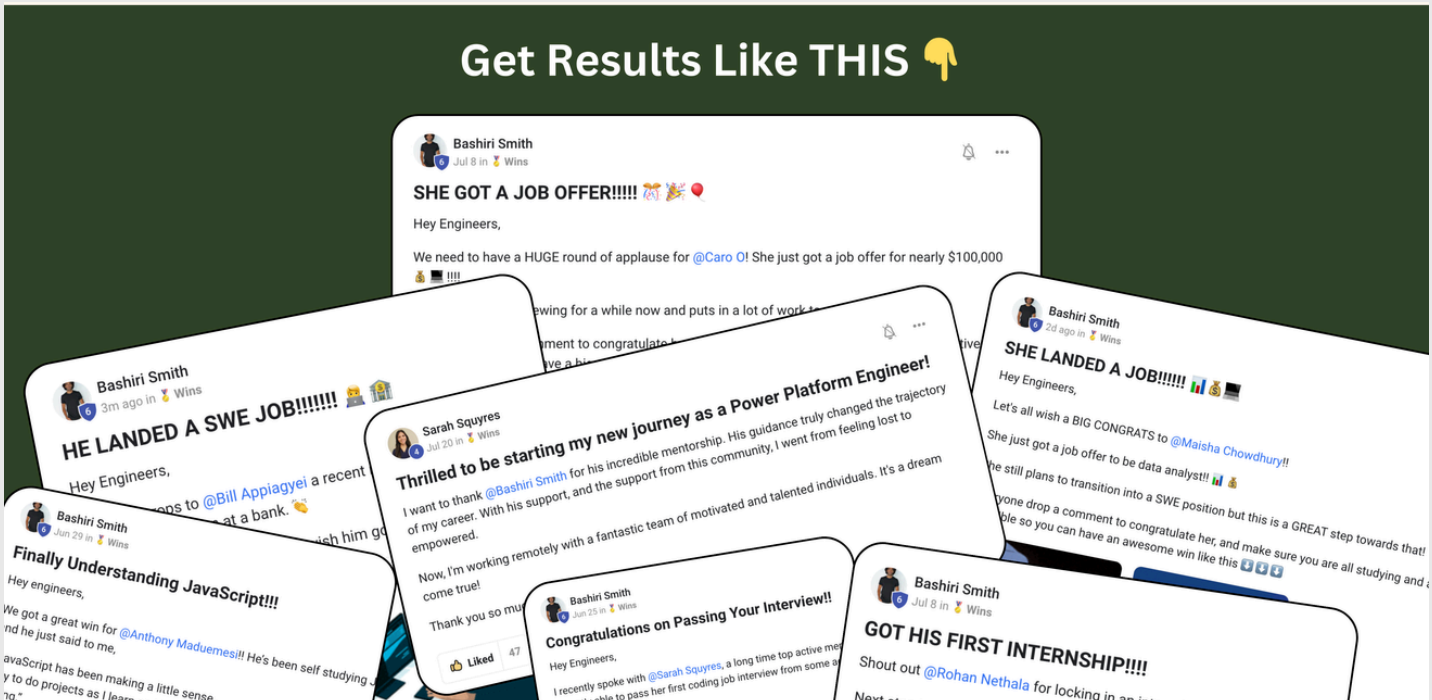
I know learning how to code on your own can be extremely daunting and just make you feel so lost. I felt the same way while I was on my journey. So now I help aspiring coders who feel lost land their first $100,000+ coding job!

This mentorship is ONLY for serious people who want to land a job ASAP.

If this sounds like you we can set up a call!

On the call, you and I will come up with a custom plan of action personalized for you so you can see exactly how to become a $100,000+ software engineer.

This call is completely FREE and is guaranteed to help you get crystal clear on your next steps.

Fill out the form below and I will reach out to you ASAP!

https://forms.gle/8LrEjeho39KvrPGG9